

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

print("Libraries imported successfully!")
```

Libraries imported successfully!

```
uploaded = files.upload()

image_path = list(uploaded.keys())[0]
print("Image uploaded:", image_path)
```

batman1.jpg

batman1.jpg(image/jpeg) - 116807 bytes, last modified: 9/15/2026 - 100% done
Saving batman1.jpg to batman1.jpg
Image uploaded: batman1.jpg

```
# Read image using OpenCV
image = cv2.imread(image_path)

# Check whether image was loaded successfully
if image is None:
    print("Error: Image could not be loaded.")
else:
    print("Image loaded successfully!")

# OpenCV reads image in BGR format
plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")
plt.show()
```

Image loaded successfully!

Original Image



```
from google.colab.patches import cv2_imshow

cv2_imshow(image)
```



```
height, width, channels = image.shape

print("Image Properties")
print("-----")
print("Width:", width, "pixels")
print("Height:", height, "pixels")
print("Number of Channels:", channels)
print("Image Resolution:", width, "x", height)
print("Data Type:", image.dtype)
print("Total Pixels:", width * height)
```

```
Image Properties
-----
Width: 1200 pixels
Height: 1200 pixels
Number of Channels: 3
Image Resolution: 1200 x 1200
```

```
Data Type: uint8  
Total Pixels: 1440000
```

```
cv2.imwrite("output_image.jpg", image)  
cv2.imwrite("output_image.png", image)  
  
print("Image saved successfully in JPEG and PNG formats.")
```

Image saved successfully in JPEG and PNG formats.

```
from IPython.display import display  
from PIL import Image  
  
print("JPEG Image:")  
display(Image.open("output_image.jpg"))  
  
print("PNG Image:")  
display(Image.open("output_image.png"))
```


JPEG Image:



```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
plt.figure(figsize=(8, 6))  
plt.imshow(gray, cmap="gray")  
plt.title("Grayscale Image")  
plt.axis("off")  
plt.show()
```

Grayscale Image



```
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

# OpenCV stores HSV values differently from normal RGB values.
# Displaying it directly gives a visualization of the HSV channels.
plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB))
plt.title("HSV Image")
plt.axis("off")
plt.show()
```

HSV Image



```
lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)

# Convert LAB back to RGB only for visualization
lab_rgb = cv2.cvtColor(lab, cv2.COLOR_LAB2RGB)

plt.figure(figsize=(8, 6))
plt.imshow(lab_rgb)
plt.title("LAB Image")
plt.axis("off")
plt.show()
```

LAB Image



```
plt.figure(figsize=(15, 4))

plt.subplot(1, 4, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original")
```

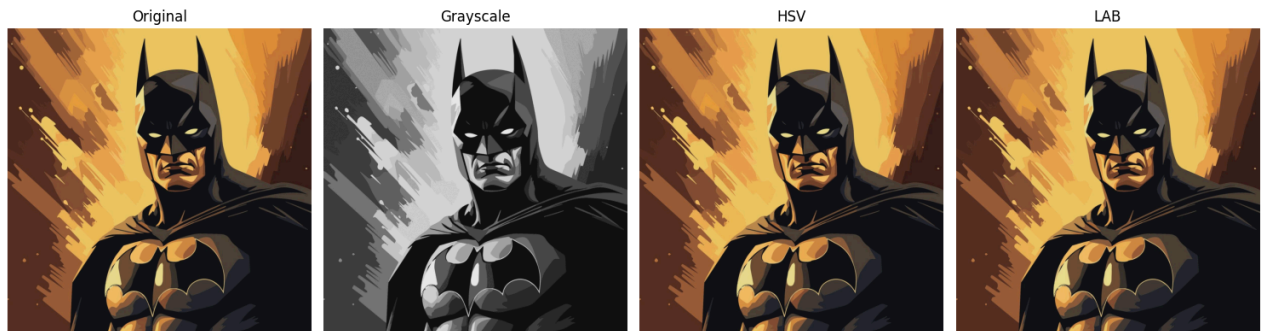
```
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(1, 4, 3)
plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB))
plt.title("HSV")
plt.axis("off")

plt.subplot(1, 4, 4)
plt.imshow(lab_rgb)
plt.title("LAB")
plt.axis("off")

plt.tight_layout()
plt.show()
```



```
# Resize to 50% of original dimensions
new_width = width // 2
new_height = height // 2

resized = cv2.resize(image, (new_width, new_height))

plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.title("Resized Image")
plt.axis("off")
plt.show()

print("Original Size:", width, "x", height)
print("New Size:", new_width, "x", new_height)
```

Resized Image



Original Size: 1200 x 1200
New Size: 600 x 600

```
# Rotate image by 90 degrees clockwise
rotated = cv2.rotate(image, cv2.ROTATE_90_CLOCKWISE)

plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title("Rotated Image - 90 Degrees")
plt.axis("off")
plt.show()
```

Rotated Image - 90 Degrees



```
# Horizontal flip
horizontal_flip = cv2.flip(image, 1)

# Vertical flip
vertical_flip = cv2.flip(image, 0)

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(horizontal_flip, cv2.COLOR_BGR2RGB))
plt.title("Horizontal Flip")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(vertical_flip, cv2.COLOR_BGR2RGB))
```

```
plt.title("Vertical Flip")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Horizontal Flip



Vertical Flip



```
negative = 255 - image

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.title("Negative / Complement Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Image



Negative / Complement Image



```
# Get image dimensions
h, w = image.shape[:2]

# Select the central region
x1 = w // 4
```

```

y1 = h // 4
x2 = 3 * w // 4
y2 = 3 * h // 4

roi = image[y1:y2, x1:x2]

plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(roi, cv2.COLOR_BGR2RGB))
plt.title("Region of Interest (ROI)")
plt.axis("off")
plt.show()

print("ROI Width:", roi.shape[1])
print("ROI Height:", roi.shape[0])
print("ROI Channels:", roi.shape[2])

```

Region of Interest (ROI)



ROI Width: 600
ROI Height: 600
ROI Channels: 3

```

plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.title("Resized")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title("Rotated")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(cv2.cvtColor(horizontal_flip, cv2.COLOR_BGR2RGB))
plt.title("Horizontal Flip")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.title("Negative")
plt.axis("off")

plt.tight_layout()
plt.show()

```